Building Web APIs with GraphQL - The Complete Guide

Section 1: Welcome

Course Introduction

1. It is becoming popular
2. Why GraphQL?
    1. Solves real problems with REST API
        1. REST is easy to use
        2. REST is platform agnostic
        3. REST enjoys great support for various development tools
        4. REST has problems too
            1. GraphQL deals with those problems
    2. Becomes super popular
        1. More systems are using it
    3. Extremely Flexible
        1. It has the capability to define what data exactly we want to retrieve is not offered by other Web APIs
    4. Backed by Facebook
        1. It will stay
    5. Leading companies are using GraphQL
        1. Facebook
        2. X
        3. Instagram
        4. Shopify
        5. The New York Times
        6. GitHub
        7. ...
3. Using GraphQL is not easy
    1. We need to take quite a few steps to use GraphQL effectively and effectively
    2. It is complex to implement and has a complex syntax (when compared to REST API)
        1. We need to be familiar with the intricacies of queries, variables, arguments, ...
    3. It requires its own libraries and dedicated code
        1. We need to use existing GraphQL API compatible with our platform
    4. There are many moving parts (we need to be familiar with them)
        1. Server implementations
        2. Syntax
        3. Resolvers
        4. ...
    5. Proprietary Error Handling
        1. It does not build on the error codes of the HTTP protocol as it is with the REST API
4. By the end of the course
    1. We'll know what is GraphQL
    2. We'll understand its advantages over REST API
    3. We'll be familiar with the GraphQL Syntax
    4. We'll know how to implement it in various platforms
    5. We'll be familiar with advanced techniques known to a few
5. And
    1. The course is extremely practical
    2. We're going to design and build a real-world library system
    3. GraphQL concepts will be explained and then used in the system
        1. At the end of the course, we will have a full-blown, fully functional GraphQL based system
    4. Using .NET and NodeJS
        1. We don't need to be familiar with the platforms
        2. We don't have to be a developer to take the course
    5. We can use it as a template for our own systems
        1. Take inspiration from it in our future GraphQL projects
    6. We'll be guided through each step and explain what it does and how to implement it

Join the Software Architects Community

Get the Course Slides

1. PDF file attached to every section's last lecture

Who is This Course For?

1. Target audience
    1. Developers
        1. Preferably back-end developers, but not mandatory
    2. Architects
        1. Can integrate the new web-api into their software architecture
    3. Anyone who is interested in web API and software architecture

Our First GraphQL Query

1. Open browser:
    1. graphql.org/swapi-graphql
        1. We can query

            1. Delete all the content

            2. Type:

```
allFilms {
    films {
        title
        director
        releaseDate
    }
}
```

1. Run
    1. It only returns only the data asked for
2. Update query:

```
allFilms {
    films {
        title
        director
        releaseDate
        directory
    }
}
```

Agenda

1. Topics:
    1. Agenda
    2. API Basics
        1. What an API is
        2. What are it's goals
    3. Web APIs
        1. A sub-set of APIs
            1. GraphQL is a Web API
        2. What is a web API?
    4. Introduction to GraphQL
        1. Why is it here?
        2. What problems does it solve?
    5. Preparing the Environment
        1. Installations
        2. Setups
    6. GraphQL Schema
        1. Most important component
            1. There is no GraphQL without it
    7. Querying Data
        1. Special syntax
    8. Mutations
        1. How we can change data in GraphQL
    9. Subscriptions
        1. Allows us to get notifications when data is changed on the server
    10. Developing GraphQL Server
        1. .NET
        2. NodeJS
    11. Developing GraphQL Client
        1. Which calls the server
    12. Advanced Topics
        1. We can improve our system by using some advanced techniques such as
            1. Error handling
            2. Authorization
    13. Conclusion

Section 2: API Basics

Read This Before Going Trough This Section

1. The section explains the basic idea of API and its goals

What is an API?

1. API: Application Programming Interface
    1. It is a set of clearly defined methods of communication among various components (software)
    2. Interfaces:
        1. User Interface
            1. What we the user sees when we look at something
                1. Websites: Homepage
            2. It is the part exposed to the user, when we try to use something
                1. User Interface - A visual representation of content we are looking at
        3. Application Interface:
            1. Interface that is exposed to the part of an application (not to an end user)
            2. We as human don't see this API
                1. Un-comprehensible to humans
            3. Example:

```
GET /account HTTP/1.1
Content-Type: application/json
Host: apigateway.us-east-1.amazonaws.com
X-Amz-Date: 20160531T184618Z
Authorization: AWS4-HMAC-SHA256 Credential={access_key_ID}/us-east-1/apigateway/aws4_request, SignedHeaders=content-type;host;x-amz-
```

4. How It Works?

```
Users Management
Software
    |
    Get user's orders
    |
    v
    [API]
Orders Management
    Software
```

1. What are the orders made by a specific user
2. Orders Management Software publishes an API
    1. API is the thing the Users Management Software can see and access to access to the internal data stored in the Orders Management Software
        1. This is called exposing the API
            1. Once it becomes public, other softwares can use it

API Types

1. API Types
    1. 4 main types of API
        1. Operating System API
        2. Library API
        3. Remote API
        4. Web API

Operating System API

1. We write some application (any platform - .NET, Java, ...)
2. The application runs inside an OS (Linux, Windows, ...)
    1. The application needs services to the OS
        1. File System - If application needs to access files, it asks the OS to give it access to the files
        2. Network Devices - If application needs to access network devices connected to the OS
            1. OS should allow it
        3. User Interface Elements - If application wants to display a button on the screen, it asks the OS a visual representation of the button
    2. How does application ask the OS?
        1. **Using OS API**
            1. All OS's exposes some kind of API
                1. The applications running on the OS can access and use the API
            2. Examples:
                1. **Win32 API** - used by all the applications running on Windows
                    1. It provides a huge number of OS services

Library API

```
My App

    Users Library
        |
        v
    [Logging API]
    Logging Library
```

```
1. When we develop an application, we use some kind of libraries
    1. The concept of libraries is very common in all the dev platforms today
2. Users library - our library
3. Logging library - Users library uses this
    1. Logging Library exposes the Logging API
        1. It allows the users library to access logging library and allows access to library methods
            1. Write to a log file
            2. Change the severity of logging
    2. Without the logging API, users library won't be able to access the logging library
4. Both users library and logging library are executed in the same process
    1. They are not communicating through network
5. Class Library
    1. .NET
    2. Java
    3. Python
    4. Node.js
    5. PHP
    6. ...
```

Remote API

```
Users Component
[Orders Proxy]
    |
    Network
    |
Orders Component
```

```
1. The two components are distributed over the network
    1. They are not in the same process
    2. They are not on the same same machine or PC
2. To communicate, they need to go over the network
    1. How?
        1. Orders Proxy
            1. Users Component builds and Orders Proxy
```

1. The proxy is used to communicate with the Orders Component
2. The proxy contains all the internal working of accessing the Orders Component
    1. The proxy knows everything there is to know about accessing the component
        1. Network protocol
        2. Network address
        3. Method signatures
        4. Authentication
        5. Authorization
        6. (everything important to access the component)
2. Using he proxy, the Users Component can access the Orders Component using a proprietary protocol
    1. Accesses methods and other functionality exposed by Orders Component
3. **It mandates that both components will be based on the same development platform**
    1. It is not possible that Users Component is based on .NET and Orders Component is based on Python
4. Example:
    1. DCOM
    2. .NET Remoting
    3. Java RMI
    4. ...
5. Remote API is using its popularity and almost no-one is using it nowadays

Web API

```
        The Internet

  Web App
     |
     Standard Protocol
     |
     v
  [Web API]
  Weather Web App
```

1. With Web API, web apps and web APIs can communicate with each other over the network
2. How does it work?
    1. Suppose we have a Web App that we developed and published over the internet
    2. The web app wants to use another web app
        1. Weather Web App
            1. It provides weather forecasts of destinations we are interested in
    3. How can web app communicate with the weather web app
        1. Steps:
            1. Weather Web App should expose a Web API
            2. Web App can access Web API
                1. Reason: Web API uses standard protocol
                    1. Using the protocol, the web api can access the weather web app for weather forecast
3. **Web API can be used across any platform or OS or Language**
    1. It doesn't matter how the web apps are implemented on both sides
        1. Web App can be developed using .NET
        2. Weather Web App can be developed using Java
    2. Hence the reason for it's success

Importance of API

1. Why do we need an API?
    1. It is very important
        1. Why?
            1. The API developed for the backend will be used:
                1. By our UI
                2. To extend our app's reach
                3. To allow monetization

API For the UI

1. Modern apps are built for web and mobile
2. Mobile apps MUST have an API on the server in order to communicate with it
    1. There is no other way for the mobile app to access data
        1. Mobile app is a separate application that runs on a separate machine on a separate process
3. Web clients built using SPA also MUST have an API on the server
    1. SPA:
        1. The frontend is developed using a sophisticated framework such as React or Angular or View
            1. They run completely independent from the server
                1. The applications are using API
                    1. HTML is rendered directly on the client (not like Web-forms or JSP)
                    2. If client needs some data from the server, it must access some kind of API
4. It allows our application to extend your reach
    1. By exposing API, other apps can use our data
        1. Outlook accesses the API exposed by forca.com and displays the data it gets back from it

Monetize

1. We can charge users for access our data
    1. Example:
        1. Twitter developer's website
            1. API is one of the most popular
                1. Standard
                    1. Certain level of API access
                2. Premium
                    1. Used to access more features or more usage for the API
                3. Enterprise
                    1. Used to access more features or more usage for the API

Why Do You Need a Well-Designed API?

1. What is use of having best practices?

1. Our developers are actually customers
    1. Whether they pay for the API or use the API freely, they are customers
        1. They have alternatives
            1. There could be other companies offering this kind of data and services
        2. Your API should be easy to use and consistent
            1. If developers have a hard time understanding what is going on in the API and how to use it
            2. Why does the API look like this for specific services and for other services, the API looks like that
                1. They might move to other customers
        3. Satisfied Customers => Less Support Calls
            1. We don't want to spend all our time and money on support centers answering questions from angry developers who don't understand what is going on and why simple things suddenly stopped working
    2. To get satisfied customers and a lot of customers and to win the hearts and money of devs and customers, we need to have a well designed and easy to use API

Section 3: Web APIs

Web APIs

1. It is the most popular and most used type of API
2. What is it?
    1. It is an API exposed by a web component, allowing other components to interact with it

Common Characteristics of Web API

1. Platform Agnostic

    ```
    .NET --------> [Web API] Python
    ```

    1. .NET component can easily communicate with Python component using Web API
2. Use standard protocols (usually over HTTP)
3. Use request/response

    ```
    .NET -HTTP Request---> [Web API] Python
          <-HTTP Response--
    ```

    1. .NET component sends an HTTP based request to a Python component
    2. .NET component will receive an HTTP based response

Web API Types

1. Differentiated mainly by:
    1. Request Format
        1. JSON
        2. XML
        3. ...
    2. Request Contents
        1. What is the difference in contents sent over the wire
    3. Response Format
        1. JSON
        2. XML
        3. ...
    4. Response Contents
        1. What is the content of response sent by server
2. Types:
    1. SOAP
    2. REST
    3. GraphQL
    4. gRPC

SOAP

1. Stands for: Simple Object Access Protocol

    1. Designed in 1998 for Microsoft
    2. XML-Based
    3. RPC style request response
        1. When a client calls a server, it calls a specific method (procedure)
            1. Remote Procedure Call
    4. It is extensible
        1. There is a base protocol that can be used to call a method and get a response
        2. It could be extended for various attributes
            1. Authentication
            2. Routing
            3. Transaction
            4. Federation
            5. ...

2. Example:

    ```
    <?xml verion="1.0"?>
    <soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope" soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
        <soap:Body xmlns:m="http://www.stock.org/stock">
            <m:GetStockPrice>
                <m:StockName>IBM</m:StockName>
            </m:GetStockPrice>
    ```

```
    </soap:Body>
</soap:Envelope>
```

1. Method: `GetStockPrice`
2. Parameter: `StockName`
    1. Value: `IBM`

3. It is quite outdated

4. We should generally not use it, unless we really have to

    1. Example: If we need to work with legacy systems that expose only SOAP API

REST

1. REST - Representational State Transfer
    1. The API is built around entities, not operations
        1. We don't call specific functions or methods
            1. We are going to define operations around entities
                1. Creation
                2. Deletion
                3. Updation
                4. Reading
    2. Designed in 2000 by Roy Fielding
    3. Built on the HTTP standard
        1. It doesn't invent new standard
            1. No new response codes
            2. No new errors
        2. It re-uses HTTP standard used by all browsers
    4. It is a message-style API
        1. It doesn't call specific methods
            1. It sends messages to the server and trusts the server to take care of the messages
    5. It is extremely simple to implement
    6. It enjoys wide support from all dev platforms
        1. It has become the most popular Web API
2. It uses URL for entity identification:

    1. `/api/v1/order/17/items`

        1. It identifies items of item number 17
        2. We send the URL to server

    2. Usually returns JSON:

```
HTTP/1.1 200
status: 200
Date: Fri, 02 Aug 2019 09:06:13 GMT
Content-Type: text/plain
Transfer-Encoding: chunked
Connection: keep-alive
access-control-allow-origin: *
vary: Accept-Encoding

{ "orderId" : "17", "orderDate" : "12.12.2018", "items" : [ "id" : "6", "id" : "56" ] }
```

Structure of REST API Response

1. Status: Indicates whether the request succeeded or not
    1. 200 - success
2. Headers: Contains meta-data about the response
3. Body: JSON with actual data sent back to the client

GraphQL

1. From the next section...

gRPC

1. It was developed by Google in 2015

    1. It is based on HTTP/2
        1. It allows two-way streaming
            1. Duplex messaging
                1. Server can send messages to the client
    2. It uses Protobuf
        1. Transport mechanism
        2. It is a serialization protocol and it allows .NET or Java to communicate between them using code-generators that read the protobuf protocol message and generate classes compliant with the data
    3. Since gRPC is based on HTTP/2, it supports bi-directional & streaming messaging
        1. Neither REST nor GraphQL supports

2. Example:

```
service HelloService {
    rpc SayHello (HelloRequest) returns (HelloResponse);
}

message HelloRequest {
    string greeting = 1;
}
```

```
message HelloResponse {
    string reply = 1;
}
```

1. Simple object

3. gRPC takes care of serialization and deserialization

4. Advantages:

    1. Quite performant - as compared to GraphQL
        1. Since it is based on HTTP/2 (very performant protocol)

5. Drawbacks:

    1. It is not widely used yet
        1. Since HTTP/2 is still not very popular
    2. Requires specilized libraries at both ends because of protobuf
        1. We need code generators at both the ends to serialized and deserialize in the platforms we are using

Section 4: Introduction to GraphQL

Problems with REST

1. Problems:
    1. During the years it was that there are some major problems with REST:
        1. **Fixed entity structure**
        2. **Request-response only**

Fixed Entity Structure

1. With REST, the client cannot specify the entity parts it wants to retrieve
2. The entity returned is set by the backend developer and cannot be modified on a per-query basis
3. Example:

    1. Employ entity:

```
{
    "emp_id": 4545,
    "full_name": "David Jones",
    "primary_phone": "938034343",
    "secondary_phone": "053403343",
    "sreet_address": "Fifth Ave. 42",
    "city": "NYC",
    "country": "USA",
    "email": "david.j@ourcamp.com",
    "birth_date": "23/01/1984",
    "base_salary": "93843"
}
```

        1. The employee is displayed in two places in the app:
            1. List of employees page
                1. We want emp_id, fullname, and email_address
                    1. 3 fields
            2. Employee page
                1. We want to see more details
                    1. Primary phone
                    2. City
                    3. Country
                2. 6 fields
        2. With REST there's no way to retrieve only the fields we need
        3. Increases load time and network latency
        4. Sometimes forces developer to construct specialized entity types just for specific queries
            1. Employee
            2. EmployeeShort (only subset of the fields)
                1. Bad for maintainenance and data model (duplicate data)

Request-Response Only

1. REST supports only the Request-Response pattern
    1. If a client sends a request to REST API, the service sends a response
2. Modern web apps require more types of client-server communication
    1. Notifications
        1. Push notifications are becoming extremely useful and popular

            1. We cannot use the request, response pattern

                1. We want service to push data to the client (not request/response)

```
Notification:
{"push_date":"2002-07-10",...}
```

            2. How to implement push notification?

                1. Can be implemented using:
                    1. Polling: from time-to-time, we send request to service and check for new notifications
                    2. Web Sockets
                        1. It is not part of the protocol
                        2. Quite complex to implement

Solution to the Two Problems

1. New API type: GraphQL

History of GraphQL

1. Facebook developed GraphQL as a replacement of REST API for internal use
    1. Due to problems
2. First used in 2012
3. Specification was open sourced in 2015
4. Currently managed by the GraphQL Foundation
    1. It is an independent body, managing all the aspects of the GraphQL specification

GraphQL Basics

1. What is GraphQL:
    1. It is a **specification**
        1. Not implementation
    2. It defines **structure of data returned**
    3. It is **JSON-based**
    4. It has **3 types of operations**
    5. It is **Schema-based**
    6. It is **Cross-platform**

A Specification

1. GraphQL defines the semantics and components of a GraphQL API
2. It does not provide concrete implementations
3. Other parties develop implementations in many languages
4. Full specs can be found here:
    1. https://github.com/graphql/graphql-spec

Defines Structure of Data Returned

1. Perhaps the most important feature of GraphQL
2. Advantages:
    1. We can specify the part(s) of entity to return (as part of the query)
    2. We can specify related entities to be returned
    3. We can specify filtering in the query
3. It solves the fixed entity structure problem
4. With GraphQL, we can specify which fields of the employee entity we can retrieve
    1. We can retrieve different fields for the:
        1. List of employees page
        2. Employee details page

JSON Based

1. GraphQL uses JSON extensively
    1. Query sent to server in JSON
    2. Data is returned in JSON
2. Technically, GraphQL can be used with other protocols
    1. However, nobody does that

3 Types of Operations

1. With GraphQL we can:
    1. Retrieve Data
        1. The most common use (by far)
        2. This is what GraphQL is used for
    2. Write / Change Data
    3. Subscribe to Changes in Data

Schema-Based

1. GraphQL works with schema
    1. Schema defines entities, fields, and attributes that we are going to use
        1. Comprises of model that is part of the system
2. Operations are built on the schema

    1. Example:

```
type Character {
    name: String!
    appearsIn: [Episode!]!
}
```

        1. We can query for the type and add or change characters in the Database

Cross-Platform

1. The client and server can be based on different platforms
2. There are many implementations in many platforms
    1. Language support of GraphQL
        1. JavaScript
        2. Go

3. PHP
4. Java / Kotlin
5. C# / .NET
6. Python
7. Swift / Objective-C
8. Rust
9. Ruby
10. Elixir
11. Scala
12. Flutter
13. Clojure
14. Haskell
15. C / C++
16. Elm
17. OCaml / Reason
18. Erlang
19. Julia
20. R
21. Groovy
22. Perl
23. D

2. Up-to-date language support list: https://graphql.org/code/#language-support

Working with GraphQL

1. There are four main steps:
    1. Defining Schema
        1. Entities & fields
            1. Together define components and models of the system
    2. Defining Queries
        1. What we can query from the schema
    3. Defining Mutations and Subscriptions (optional)
    4. Implementing the Logic
        1. Of queries, mutation, and subscriptions

Section 5: Preparing the Environment

Introduction

1. In order to build the app, we need to install some software on our computer
2. We'll Install:
    1. .NET SDK
    2. NodeJS
    3. VS Code with some extensions
        1. Which we will use as a development tool for our app
3. Note:
    1. We don't have to be a developer in order to build the app
        1. We'll go through all the steps

Install .NET SDK

1. Search: Download .NET SDK
    1. .NET SDK x64
    2. .NET 6.0 LTS
2. Open command-prompt
    1. `dotnet --version`

Install NodeJS

1. URL: nodejs.org/en/download
    1. Install
2. CMD:
    1. `node --version`

Install VS Code

1. Search: Download VS Code
    1. Select OS: Windows
        1. System installer
2. Install
3. Pin to taskbar
4. Extension
    1. C#
        1. C# by Microsoft

Download and Run the Demo App

1. Download file from resources section
    1. Extract in `graphql_demo`
        1. Right click
        2. Open with Code
2. Run the project
    1. F5

1. GraphQL playground
    1. Hot Chocolate - for .NET
        1. There is a playground for each platform
    2. Click Create Document
        1. Schema endpoint: Points to the GraphQL endpoint (created by default by our GraphQL implementation)
            1. Every GraphQL implementation has its own endpoint
                1. Usually named: `graphql`
            2. Apply
    3. Type:

```
{
    books {
        name
        pages
    }
}
```

        1. Gives results
        2. Modification:

```
{
    books {
        bookId
        name
        pages
    }
}
```

            1. Works
        3. books.json
            1. Database
            2. Add the following:

```
{
    "bookId": 91,
    "name": "It ends with us",
    "pages": 384
}
```

                1. Run the query

Section 6: GraphQL Schema

Introduction

1. GraphQL Schema:
    1. The first step when working with GraphQL is to define the schema
    2. GraphQL cannot work without schema

Role of Schema

1. With GraphQL we can query specific fields from specific objects:
    1. Solution: GraphQL needs to know there is such an object and such fields:
        1. The book object
        2. name and pages fields
    2. The Schema is used to:
        1. Define the shape of data
        2. Define queries that we can run against the data
        3. Define mutations
        4. Define subscriptions
2. GraphQL uses the schema to:
    1. **Validate queries**
        1. When a query is run against GraphQL, GraphQL can use schema to ensure the fields included in the query are actually fields that exist in the object
    2. **Validate other operations**
        1. Not just queries
    3. **Provide auto-complete in playground**

Schema Type System

1. The schema uses type system to define:
    1. **Field name**
    2. **Field type**
    3. **Nullability**
        1. Whether the field can contain null values
2. Schema Language
    1. The schema uses a specialized language called
        1. **Schema Definition Language** (SDL)
            1. The language is used to define:
                1. Objects
                2. Fields
                3. Type system

4. Queries
5. ...

3. How do we generate the scheme?
    1. The schema is sometimes auto-generated by the GraphQL implementation
        1. It is based on the concrete language-specific objects
            1. .NET - Schema is created automatically by the GraphQL implementation
            2. Node.JS - We need to manually define the schema
    2. The schema can be viewed in the playground
4. Playground:
    1. Schema Reference
        1. Easier way to view and navigate
            1. Objects
                1. Two objects
                    1. Book
                        1. Fields
                            1. bookId
                            2. name
                            3. pages
                    2. Query
                        1. It is an object with field
                            1. Books
                                1. Three fields
    2. Schema Definition

        1. Full definition of the schema in JSON format

```
type Query {
    books: [Book!]!
}

type Book {
    bookId: Int!
    name: String!
    pages: Int!
}
```

Object Type and Fields

1. They are the basic building blocks of schema

    1. They describe entities (=>Objects) and their properties (=>fields) in the system
    2. They allow GraphQL to be familiar with the system's objects and to provide validation to these entities (objects)

2. Schema of our app:

```
type Book {
    bookId: Int!
    name: String!
    pages: Int!
}
```

    1. Object type: Book
    2. Fields:
        1. bookId
        2. name
        3. pages
    3. Each field has a data type (indicates the type of data it can store)
        1. GraphQL specification contains built-in scalar types
            1. Scalar types: primitive types that hold a single scalar value
        2. Complex types can also be defined
            1. They are not part of the spec

Scalar Types

1. Built in scalar types:
    1. Data Type
        1. `Int`: **(M)** A signed 32-bit integer (64, 5890, -17 etc)
        2. `Float`: **(M)** A signed double precision floating-point value (18.99 etc)
        3. `String`: **(M)** A string of characters UTF-8 encoded ("Hello world" etc)
        4. `Boolean`: **(M)** `true` or `false`
        5. `ID`: **(M)** A unique identifier, identical to String
            1. It is used to indicate to the user that this field is a unique identifier
            2. Internal value of ID is String
2. Most implementations add customer scalar types:
    1. Examples:
        1. `date`
3. Objects and fields in action:

    1. Models > Book.cs - defines the schema and format of the book object

        1. Schema is built based on this file

    2. Add the following field:

```
public double Price {get; set;}

public DateTime PublishDate {get; set;}
```

        1. Restart the app

2. Go to schema definition

```
price: Float!
publishDate: DateTime!

scalar DateTime
```

1. `DateTime` - it was added by this specific implementation
    1. It is not part of the GraphQL scalar types

Adding Object Type

1. New object type:
    1. Step 1: Define new object in code
        1. Code we are familiar with
    2. Step 2: Include it in GraphQL Model
2. VS Code
    1. Explorer:
        1. Models > Author.cs

```
public class Author {
    public int AuthorId {get; set;}
    public string Name {get; set;}
}
```

1. Books.cs

```
...
public Author? Author {get; set;}
```

1. `?` - This field is optional

2. Books.json

```
{
    ...
    "author": {
        "authorId": 108,
        "name": "Abdul Kalam"
    }
}
```

3. Restart the app

2. Query:

```
{
    books {
        bookId
        name
        pages
        author {
            name
        }
    }
}
```

Nullable Fields

1. There's an exclamation mark near the type of fields in the schema

```
type Book {
    bookId: Int!
    name: String!
    pages: Int!
    price: Float!
    publishDate: DateTime!
}
```

1. `!` - Indicates that the field is non-nullable - it can't be null
    1. If we'll try to construct a new Book object using GraphQL and don't put values in all the fields - you'll get an error
        1. Can be changed using the object definition in the code

2. VS Code
    1. Book.cs

```
public DateTime? PublishDate {get; set;}
```

1. Restart the app
    1. Schema definition

```
publishDate: DateTime
```

1. No `!`

Enumeration Type

1. A special type of scalar
    1. It limits the values a field can have to a predefined list
        1. Great for making sure only valid values are selected

2. Invalid values will be marked as error in GraphQL

2. Book.cs

```
public class Book {
    ...
    public DateTime? PublishDate {get; set;}
    public BookGenre Genre {get; set;}
    ...
}

public enum BookGenre {
    Horror,
    Fantasy,
    Drama,
    Thriller,
    NonFiction
}
```

1. Schema Definitions

```
type Book {
    ...
    genre: BookGenre!
    ...
}

enum BookGenre {
    HORROR
    FANTASY
    DRAMA
    THRILLER
    NON_FICTION
}
```

2. books.json

```
{
    ...
    "genre": "Fantasy"
    ...
}
```

3. Playground:

```
{
    ...
    genre
    ...
}
```

1. Default value is used which is the first value

Lists

1. Objects can hold lists of other objects
2. Can be declared in the schema
3. Example: Reviews per book
    1. VS Code
        1. Models > Review.cs

```
public class BookReview {
    public int ReviewId {get; set;}
    public int Rating {get; set;}
    public string Description {get; set;}
    public DateTime ReviewDate {get; set;}
    public string User {get; set;}
}
```

2. Models > Book.cs

```
...
public BookReview[] Reviews {get; set;}
```

3. Restart the app
    1. Schema definition

```
reviews: [BookReview!]!
```

1. Two `!`
    1. `BookReview!` - The list cannot contain null items
        1. Everything in the list must be a non-null value
            1. We cannot push `null` into the list
    2. `[..]!` - There has to be a list (even an empty one). The `reviews` field cannot be null

2. Modify the query:

```
{
    books {
        ...
        reviews {
            rating
            description
        }
    }
}
```

1. Null cannot be returned

3. Book.cs

```
public BookReview[]? Reviews {get; set;}
```

4. `null` is returned

4. Books.json

```
{
    ...
    "reviews": [
        {
            "reviewId": 54,
            "rating": 5,
            "description": "A real page turner",
            "user": "Meal"
        }
    ]
}
```

5. Run the query

Interfaces

1. Similar to modern development languages, GraphQL allows defining interfaces
    1. Interface: An abstract type with a set of fields
2. In order to implement interface, a type must include this set of fields
    1. Allows working with multiple types using a single interface
3. VS Code

    1. Models > IReadingMaterials.cs

    ```
    public interface IReading Materials {
        string Name {get; set;}
        BookGenre Genre {get; set;}
    }
    ```

    2. Models > Book.cs

    ```
    public class Book : IReadingMaterials {
        ...
    ```

    3. Models > Program.cs

    ```
    builder.Services.AddGraphQLServer()
        .AddQueryType<Query>()
        .AddInterfaceType<IReadingMaterials>();
    ```

    4. Run the app

        1. Schema Definition

        ```
        interface IReadingMaterials {
            name: String!
            genre: BookGenre!
        }

        type Book implements IReadingMaterials {
            bookId: Int!
            name: String!
            pages: Int!
            price: Float!
            publishDate: DateTime
            genre: BookGenre!
            author: Author
            reviews: [BookReviews!]
        }
        ```

            1. When querying for IReadingMaterials, we can return Books
                1. To use it, we must define another object type implementing the interface

                1. Models > Magazine.cs (when retrieving reading materials, we will get Magazines and Books

                ```
                public class Magazine : IReadingMaterials {
                    public string Name {get; set;}
                    public BookGenre Genre {get; set;}
                    public int InssueNo {get; set;}
                }
                ```

                2. Database > magazines.json

                ```
                [
                    {
                        "magId": 117,
                        "name": "Car and Driver",
                        "genre": "NonFiction",
                        "IssueNo": 18
                    }
                ]
                ```

                3. Query.cs

                ```
                ...
                public List<Magazine> Magazines => ReadMagazines(); // exposes the list of objects to query object
                ...
                prvate List<Magazine> ReadMagazines() {
                ```

```
    string fileName = "Database/magazines.json";
    string jsonString = File.ReadAllText(fileName);
    return JsonSerializer.Deserialize<List<Magazine>>(jsonString, new JsonSerializeOptions { PropertyNameCaseInsens
}
```

4. Restart the app

    1. Schema definition

    ```
    type Query {
        books: [Book!]!
        magazines: [Magazine!]!
    }
    ```

5. Operations:

```
{
    magazines {
        name
    }
}
```

6. Query.cs

```
public List<IReadingMaterials> ReadingMaterials => GetReadingMaterials();
...
private List<IReadingMaterials> GetReadingMaterials() {
    var materials = ReadBooks().Cast<IReadingMaterials>().ToList();
    materials.AddRange(ReadMagazines().Cast<IReadingMaterials>().ToList();
    return materials;
}
```

    1. Helps in querying IReadingMaterials which returns both Books and Magazines

    2. Restart the app

    3. Query:

    ```
    {
        readingMaterials {
            name
            genre
        }
    }
    ```

        1. Returns both books and magazines

Unions

1. They are similar to interfaces
    1. They allow querying multiple object types in a single query
    2. In contrast to interface, object types do not have to have common fields
    3. Query can specify which field to retrieve from each object type
2. Unions in action:

    1. Step 1: Define interface without any members

        1. Marker interface
            1. It indicates that all implementing members will be part of the union
        2. Example:

            1. Models > IThings.cs

            ```
            [UnionType("Things")]
            public interface IThings {}
            ```

                1. `[UnionType("Things")]` - the interface is of type union and it is named as `Things`

    2. Step 2: Define the classes as implementations of the interface

        1. Example:

            1. Book.cs

            ```
            public class Book: IReadingMaterials, IThings
            ```

            2. Magazine.cs

            ```
            public class Magezine: IReadingMaterials, IThings
            ```

    3. Step 3: Add a method to return List of IThings

        1. Example:

            1. Query.cs

            ```
            ...
            public List<IThings> Things => GetThings();
            ...
            private List<IThings> GetThings() {
                var things = ReadBooks().Cast<IThings>.ToList();
                things.AddRange(ReadMagazines().Cast<IThings>().ToList());
                return things;
            }
            ```

            2. Schema definition:

```
type Query {
    ...
    things: [Things!]!
}

union Things = Magazine | Book
```

3. Operations:

```
{
    things {
        __typename
        ... on Book {
            name
            publishDate
            reviews {
                rating
            }
        }
        ... on Magazine {
            name
            genre
        }
    }
}
```

1. `__typename` - Used to specify which fields we want to retrieve per type
    1. Reason: There aren't any common fields defined by the interface (unlike `interface`)

Section 7: Queries

Introduction

1. GraphQL Queries
2. The second step in working with GraphQL is running the query
    1. Setting up the schema is usually a one-time step (with some modification later)
        1. The whole purpose of it is to run queries against it
            1. The most important part of GraphQL

Query Basics

1. We already played with queries when defining schema
2. We saw we can use it to query:
    1. Object types
    2. Fields
    3. Related objects
3. Quick refresher
    1. Run the app
    2. Open the playground

        1. Operations

```
{
    books
}
```

        1. Error

            1. We need to specify the actual fields to retrieve

        2. Hit space to get a list of fields

```
{
    books {
        bookId
        name
        genre
        pages
    }
}
```

        3. Selecting related entities

```
{
    books {
        bookId
        name
        genre
        pages
        author {
            authorId
            name
        }
    }
}
```

Operation Type and Name

1. We didn't specify that we actually query the data
    1. We just typed in what we want to fetch and let GraphQL do its thing
        1. GraphQL inferred that what we want to do is to query data and not, for instance, construct an entity
2. It's a good practice to specify the **Operation Type** when calling GraphQL

    1. Reason: It makes it clear what we want to do

2. Plaground:

```
query {
    books {
        ...
    }
}
```

    1. We specified that we want to query the data
        1. If we don't specify the operation type, it just assumes that we want to query the data
    2. It is a best practice to specify the operation type

Operation Name

1. When calling GraphQL with multiple queries, it's a good practice to name the query
    1. Reasons: It helps when debugging and logging the query on the server side
2. Simple to implement

    1. **Add the name after the query keyword**

```
query GetMyBooks {
    ...
}
```

    1. Result:
        1. Bottom part:
            1. List of queries: Shows the name of the query run

Arguments

1. So far, when querying, we asked to return all the records
    1. Nice for learning purposes but not practical
        1. We usually want to retrieve some of the data, and be able to specify how we want to filter it
            1. Solution: **Query arguments**
2. VS Code:

    1. Query.cs - to query using parameters/arguments

```
// public List<Books> ...

public List<Book> Books(string nameContains = "") { // we go straight to the method and not through variable we defined above to specify a
    string fileName = "Database/books.json";
    string jsonString = File.ReadAllText(fileName);
    var books = JsonSerializer.Deserializer<List<Book>>(...);
    return books.Where(b => b.Name.IndexOf(nameContains) >= 0).ToList();
}
```

    1. Restart the app

        1. Schema Definition

```
type Query {
    books(nameContains: String) = ""): [Book!]!
    ...
}
```

        2. Query:

```
query GetByBooks {
    books (nameContains: "M") {
        ...
    }
}
```

Variables

1. In the arguments we defined, we included the value of the argument in the query

    1. Usually we want to specify the value separately from the query string
        1. Solution: We can use variables
            1. We do not require any code changes for this

2. Example:

```
query GetMyBooks ($bookName: String) {
    books (nameContains: $bookName) {
        ...
    }
}
```

    1. We can go to **Variables** section in playground

```
{
    "bookName": "H"
}
```

Aliases

1. We can include queries in a single query call
2. We can also query the same object type multiple times in the same query call
    1. This can cause a conflict in the result, as the result is named after the object type queried
3. Example:

    1. Query:

```
query GetMyBooks {
    books (nameContains: "H") {
        bookId
        name
        genre
    }
}
```

1. Name of the result: "Books"

2. Another query:

```
query GetMyBooks {
    books (nameContains: "H") {
        bookId
        name
        genre
    }
    books (nameContains: "M") {
        bookId
        name
        genre
    }
}
```

1. Error: We have a set of fields with same object type that cannot be merged
    1. We ran duplicate queries for same object type which caused a conflict
        1. The conflict can be solved using aliases

Aliases

1. With aliases, we give a unique name to each query in the request
    1. The result is named after the alias (not after the object type)
        1. We can avoid conflicts
2. Example:

```
query GetMyBooks {
    booksWithH : books (nameContains: "H") {
        ...
    }
    booksWithM : books (nameContains: "M") {
        ...
    }
}
```

1. Query names in result will be aliases

Fragments

1. In the previous example, we say two queries
    1. The only difference between them is the argument
    2. They both queried for the same fields
        1. This can be repetitive and error prone
            1. Solution: Fragments allow us to define the set of fields to retrieve and use it in queries
2. Query

```
query GetMyBooks {
    booksWithH: books (nameContains: "H") {
        ...bookFields
    }
    booksWithM: books (nameContains: "M") {
        ...bookFields
    }
}

fragment bookFields on Book {
    bookId
    name
    genre
}
```

1. `on` - assigns the gragmet to an object type
2. Fragments are used to specify the fields to retrieve

3. Specifying unique fields for each query in addition to the fragment

```
booksWithH: books (...) {
    ...bookFields
    author {
        name
    }
}
```

Directives

1. So far a query always returned the same set of fields
2. We want query to return different set of fields based on a specific scenario
    1. Example: For a list of entities, we would need less fields than a detailed view of the entity
        1. This can be achieved through directives

Directives

1. With directives, we can dynamically change the structure and shape of the query based on variables passed to the query
2. Each directive checks whether a given argument is `true` and then executes its instructions
3. Currently two directives are part of GraphQL specification:
    1. `@include (if: Boolean)` **(M)** - Include the field if argument is `true`
    2. `@skip (if: Boolean)` **(M)** - Skip a field if argument is `true`
4. Example:

    1. Playground

```
query GetMyBooks ($withAuthor: Boolean!) {
    ...
    author @include(if: $withAuthor) {
        ...
    }
}
```

        1. Variables

```
{
    "bookName": "L",
    "withAuthor": true
}
```

        2. Variables

```
{
    ...
    "withAuthor": false
}
```

Variables in Fragments

1. Example:

```
query GetMyBooks ($withAuthor: Boolean!, $noGenre: Boolean!) {
    ...

}

fragment bookFields on Book {
    ...
    genre @skip (if: $noGenre)
}
```

    1. Variables:

```
{
    ...
    "noGenre": false

}

{
    ...
    "noGenre": true
}
```

Inline Fragments

1. We can use interfaces to retrieve multiple object types in a single query
2. What if we want to retrieve specific type fields even when querying interfaces
    1. Solution: Inline fragments

Inline Fragments

1. We define which fields whould be retrieved in addition to the fields included in the interface
    1. Use the ...type syntax
        1. Similar to regular fragments
2. Example:

    1. New tab in the playground

```
{
    readingMaterials {
        name
        ...on Book {
            pages
        }
        ...on Magazine {
            issueNo
        }
    }
}
```

Meta Fields

1. In the previous example, we saw how we can retrieve multiple object types in a single result
    1. It can useful if the query will exactly tell us what is the object type of each item in the results
        1. Solution: Meta Fields

Meta Fields

1. Meta fields provide data about the types returned
    1. They are part of **Introspection** mechanism

2. We will learn about introspection next
3. The most useful meta field is `__typename` field
    1. It indicates the name of the object type returned

        1. Example:

```
{
    readingMaterials {
        __typename
        name
        ... on Book {
            pages
        }
        ... on Magazine {
            issueNo
        }
    }
}
```

            1. Result contains:

```
"__typename": "Book"
```

                1. The object type is returned

Introspection

1. When we start working with a schema of a GraphQL server, we don't always know what are the objects in the schema and what fields do they contain

    1. Solution: **Introspection**
        1. It allows us to explore the schema and learn about its objects and fields
        2. It is used as a query with GraphQL keywords

2. Example:

    1. New tab in playground

        1. Operations:

```
{
    __schema {
        types {
            name
            kind
            description
            fields {
                name
                type {
                    name
                    kind
                    ofType {
                        name
                    }
                }
            }
        }
    }
}
```

            1. Returns all types including built-in types
            2. `kind` - OBJECT, UNION, ...

3. This is a way to learn quickly about the GraphQL server if we don't have the documentation

4. To see schema of a specific type (instead of all the types)

```
{
    __type (name: "Book") {
        name
        kind
        description
        fields {
            name
            type {
                name
                kind
                ofType {
                    name
                }
            }
        }
    }
}
```

Section 8: Mutations

Introduction

1. Most uses of GraphQL are around queries
    1. GraphQL can also be used to change data
        1. Called **Mutation**

Mutation Syntax

1. There are three main parts in the mutation request:

1. **Mutation Object Type**
2. **Mutation Parameters**
3. **Returned Data**
4. Example:

```
mutation ($book: BookInput!) {
    addBook (input: $book) {
        bookId
    }
}
```

    1. `mutation` - object type
        1. Defines that we're running a mutation and not a query
    2. (`$book: BookInput`) - Arguments that we want to be created/modified (can be multiple)
        1. `BookInput` - Special input type
    3. `bookId` - Returned data
        1. Reflects the new object that was added
            1. In query: Field name is the field that we want to query
            2. With mutation: Fields we want to get back after inserting or mutating an object
    4. Mutations receive a special input named **Input Type**

Input Types

1. When using queries, we usually use scalar argument:
    1. BookId => Int
    2. BookName => String
    3. ...
2. With mutations, we usually want to pass complex argument containing **properties of the entity we want to construct**
    1. This type of argument is called **Input Type**
        1. It can be used in mutations, but also in queries
        2. It **needs to be defined in code and then be used in the mutation**
    2. **Every complex type can be used as input type**
3. VS Code:
    1. Models > BookInput.cs

        1. It contains input type that will be used to add a new Book

```
public class BookInput {
    public string Name {get; set;}
    public int Pages {get; set;}
    public double Price {get; set;}
    public DateTime? PublishDate {get; set;}
    public BookGenre Genre {get; set;}
}
```

        1. Restart the app
            1. Schema Reference
                1. Input Objects are not shown
                    1. We didn't use it in query or mutation

Adding Mutation to the Schema

1. Download mutation.zip file and open it
    1. Mutation.cs
        1. Copy it in `GraphQL` folder
2. VS Code

    1. Method: `AddBook`
    2. Parameter: `BookInput` object
    3. It retrieves a list of books
    4. It then generates a new `Book` object based on params received in input type
    5. We then add book to books list
    6. We then write the list back to the file

        1. We can use database instead

    7. We return a the new book
    8. Program.cs

```
builder.Services.AddGraphQLServer().
    AddQueryType<Query>().
    AddInterfaceType<IReadingMaterials>().
    AddMutationType<Mutation>();
```

    1. Run the app or restart it
    2. Playground
        1. Schema Reference
            1. Mutation type
                1. AddBook method
                    1. It receives BookInput
            2. Input Object
                1. BookInput

1. It has fields

Using Mutation

1. Playground > New tab

```
mutation ($book: BookInput!) {
    addBook (input: $book) {
        bookId
        name
        pages
    }
}
```

    1. `addBook` method
        1. `$book` - variable of type `BookInput`
    2. `bookId`, `name`, `pages` are returned after the mutation
    3. Variables

```
{
    "book": {
        "name": "Rich Dad Poor Dad",
        "pages": 274,
        "genre": "NON_FICTION",
        "price": $4.99
    }
}
```

    4. Run
        1. Returns the details of the new book
    5. New tab:

```
{
    books {
        bookId
        name
        genre
        price
    }
}
```

        1. New book appears in the result

Section 9: Subscriptions

Introduction

1. It allows us to subscribe to events in the GraphQL server
2. When an event is triggered, the subscription gets a notification
    1. It is usually used in conjunction with Mutations
        1. When an entity is changed or added, other clients can get notifications and can act upon it
3. Subscriptions are rarely used in GraphQL

Subscriptions Syntax

1. There are three main parts in the subscription request:
    1. **Subscription Object Type**
    2. **Subscription Name**
    3. **Returned Data**
2. Example:

```
subscription {
    bookAdded {
        bookId
        name
    }
}
```

    1. `subscription` - object type
        1. Defines that we're running a subscription and not a query
    2. `bookAdded` - Subscription name
        1. The name of the subscription we want to subscribe to
    3. `bookId`, `name` - Returned data
        1. Reflects the new object that was added or modified
3. Steps:
    1. First step is to define and configure the subscription in code

Adding Subscriptions to the Schema

1. VS Code
    1. GraphQL > Subscription.cs

```
public class Subscription {
    [Subscribe]
    public Book BookAdded([EventMessage]Book newBook) => newBook;
}
```

1. `BookAdded` - subscription that clients can subscribe to

2. Program.cs

```
AddQueryType...
    .AddSubscriptionType<Subscription>();
```

1. Tells GraphQL that there is a subscription type called `Subscription` and clients can subscribe to it

3. We need to specify where the subscription data is going to be managed

    1. When a subscription is triggered, the data is stored somewhere until clients can pull it
        1. Solution: **Default memory provider**
            1. **There are other providers**

4. Program.cs

```
builder.Services.AddInMemorySubscriptions(); // use default in-memory provider
```

5. Communication method between the client and server

    1. Subscriptions are usually implemented using web-socket protocol

        1. Program.cs

```
app.MapGraphQL();
app.UseWebSockets();
```

6. Run the app (F5)

    1. Playground:
        1. Schema Reference
            1. Subscription
                1. BookAdded
                    1. Sends Book object

Using Subscriptions

1. We want to send notifications to subscribers when a new book is added
2. VS Code
    1. Mutation.cs

        1. `AddBook` method will receive a sender object

        2. The sender object will send message to subscribers

```
using HotChocolate.Subscriptions;

AddBook(BookInput input, [Service])ITopicEventSender sender) {
    ...
    // Send subscription notification about the new book
    sender.SendAsync("BookAdded", book);

    // Return the newly created book
    return book;
```

            1. `[Service]` - The parameter is a service that .NET platform is going to initialize for us

        3. Restart the app

            1. Playground

                1. New Tab:

```
subscription {
    bookAdded {
        bookId
        name
    }
}
```

                    1. Run
                        1. It is continously running
                            1. It waits for data from GraphQL server

                2. Go back to mutation tab

                    1. Click run
                        1. New book would be created

                3. Go back to subscription tab

                    1. We see a new book
                        1. It continues to wait

                4. Go back to mutation tab

                    1. Run

                5. Go back to subscription tab

1. It shows new book

6. Bottom right:

1. Shows events we got

Section 10: Developing GraphQL Server

Introduction

1. Developing GraphQL Server
    1. We learned quite a lot about GraphQL specifications and syntax
    2. We can develop a GraphQL based system
2. Two components in GraphQL - based system:
    1. GraphQL Server (.NET Project in VS Code)
        1. Exposes the GraphQL endpoint
        2. Receives the operations
        3. Returns results
    2. GraphQL Client (Playground)
        1. Sends operations to the GraphQL Server
        2. Handles the results

GraphQL Server Roles

1. GraphQL Server has the following roles:
    1. **Define GraphQL Schema**
    2. **Expose GraphQL Endpoint**
    3. **Validate GraphQL Operations**
    4. **Route GraphQL Operations to Code**
        1. Convert to code entities
    5. **Return GraphQL Results**
2. Every GraphQL server must follow GraphQL specification

Define GraphQL Schema

1. The GraphQL Server defines the schema using code elements in the server or **explicitly using the Schema Definition Language** (NodeJS Demo)
    1. It is usually based on classes / modules
    2. Each server has its own schema definition mechanism

        1. Example: .NET

```
builder.Services.AddGraphQLServer()
    .AddQueryType<Query>()
    .AddInterfaceType<IReadingMaterial>()
    .AddMutationType<Mutation>()
    .AddSubscriptionType<Subscription>();
```

        1. We have explicitly specified what our Query type, Interface Type, Mutation Type, and Subscription type are going to be
            1. It is different in Node.js

Expose GraphQL Endpoint

1. All GraphQL operations are sent to specific, single endpoint

    1. Opposed to REST API which exposes an endpoint for each operation:

```
GET /api/v1/books/{id}
POST /api/v1/employee
```

    2. In GraphQL we have a single endpoint

        1. The endpoint is usually named `graphql`
        2. The endpoint is accessed using `POST` verb

    3. The server expects to receive JSON payload in the endpoint

Validate GraphQL Operations

1. The server receives a JSON payload and validates it against the schema
    1. If a mismatch is found - the server returns a standard error message
2. No coding is required for the validation

Route GraphQL Operations to Code

1. The server converts JSON payload to objects and triggers methods to process the operations and return the data
2. The methods are called `Resolvers` (important)

    1. The server traverses through the query and runs a resolver **for every field**

        1. **We have a resolver for every field** (even though in code we have a single resolver per object type)

    2. If the field returns a scalar value, that is what's returned

    3. If the field returns an object value, the server runs resolvers for each field in the object

        1. ... until scaler is returned

4. Resolvers in the same hierarchy run in parallel
    1. Implies that one can run before the other
        1. **NEVER use data from another resolver in our resolver**
5. Example:

```
{
    books {
        name
        pages
        author {
            name
        }
    }
}
```

    1. First resolver: Resolver of the root element (for `books` object)
        1. It returns `Book` object
        2. Since `Book` is an object, additional resolvers are executed on its fields
    2. Next resolvers: Resolvers are executed for the fields `name`, and `pages`
        1. The resolver return scalar values
            1. Returned directly
    3. Next resolver: Resolver for the `author` element (which is run in parallel to the previous resolvers)
        1. Returns an `Author` object
        2. Since `Author` is an object, additional resolvers are executed on its fields
    4. Next resolver: Resolver of the `name` field
        1. Returns scalar value
3. Most implementations automatically construct resolvers for fields
    1. As in our case
        1. HotChocolate implementation automatically generates resolvers for each field
            1. **Custom resolvers can be specified**
4. **Resolvers can accept arguments with context data about the query**

Return GraphQL Results

1. After running the resolvers and merging the results, the server converts the object to JSON and returns it to the client

Configuring GraphQL Server in .NET

1. We'll develop GraphQL in two platforms:
    1. .NET
    2. NodeJS
2. We need to always use existing libraries for doing that
    1. Never develop our own implementation

Developing GraphQL Server in .NET

1. There are various libraries for GraphQL in .NET
    1. We will use one of the most popular one **HotChocolate**
2. Steps:
    1. We will construct an empty .NET web-app first and convert it into a GraphQL server
        1. GraphQLCourse > MyGraphQLNet
            1. Open cmd
            2. Empty web-app: `dotnet new web`
            3. Add Hot Chocolate GraphQL library: `dotnet add package HotChocolate.AspNetCore`
                1. Downloaded required packages for GraphQL
            4. `code .`
                1. Required assets to build and debug:
                    1. Click `Yes`
                        1. Adds some config files for allowing VS Code to debug and run .NET programs
                2. `Program.cs` - main and initial file of .NET
                    1. Configure it to use GraphQL libraries of HotChocolate
                    2. Add the following:

```
builder.Services.AddGraphQLServer();
...
app.MapGraphQL();
```

                        1. The project is fully configured to run GraphQL
            3. F5
                1. Open playground: **http://localhost:7293/graphql**
                    1. Schema Reference - Schema is unavailable

Developing the Schema in .NET

1. It is straightforward
    1. It built on regular classes
2. VS Code:
    1. New folder: Models
        1. New file: Book.cs

```
public class Book {
    public int BookId {get; set;}
    public string Name {get; set;}
    public int Pages {get; set;}
```

```
    public DateTime publishDate {get; set;}
}
```

2. Query object:
    1. New folder: GraphQL
        1. New file: Query.cs

```
public class Query {

}
```

            1. It is empty
            2. It has no resolvers
                1. We need to tell GraphQL that we need to use this class that will be used for resolving queries
    2. Program.cs

```
builder.Services.AddGraphQLServer()
    .AddQueryType<Query>();
```

        1. Indicates to GraphQL that the `Query` class has the resolvers for queries
    3. Query.cs

```
public List<Book> Books(string nameContains="") {
    string fileName = "Database/books.json";
    string jsonString = File.ReadAllText(fileName);
    var books = JsonSerilizer.Deserialize<List<Book>>(jsonString, new JsonSerializerOptions { PropertyNameCaseInsensitive = true, Co
    return books.Where(b => b.Name.IndexOf(nameContains).ToList());
}
```

        1. The resolver is used to resolve `Book` object
            1. The fields are automatically resolved to the actual fields of the Book object
    4. Database:
        1. New folder: `Database`
            1. Copy paste `books.json`
    5. Configuring VS Code to automatically open GraphQL page when running
        1. .vscode > launch.json

```
"serverReadyAction": {
    ...
    "uriFormat": "%s/graphql"
}
```

    6. Restart the app
    7. ctrl+. to fix imports
    8. Run the app
        1. Schema Reference
            1. We have a schema
        2. Operations:

```
{
    books {
        name
        publishDate
    }
}
```

            1. Run
            2. Arguments

```
query ($name: String) {
    books (nameContains: $name) {
        name
        publishDate
    }
}
```

                1. Variables:

```
{
    "name": "H"
}
```

    9. Publish date is displayed in standard format
        1. We want to define a resolver that will return a human readable date
            1. Need a special resolver class
                1. GraphQL > Resolvers.cs

```
class Resolvers {
    public string GetFormattedDate([Parent]Book e) {
        return e.PublishDate.ToShortDateString();
```

```
    }
}
```

1. `[Parent]Book` - `Book` object is the parent of the field of which we want to change the value

2. Where to use it?
    1. Query.cs

```
...
public class BookType : ObjectType<Book> {
    protected override void Configure(IObjectTypeDescriptor<Book> descriptor) {
        descriptor.Field("publishDate").ResolveWith<Resolvers>(r => r.GetFormattedDate(default!));
    }
}
```

        1. The code defines an object type (GraphQL type for `Book` class we defined)
            1. This enables customization of GraphQL
                1. Customization:
                    1. Configure a resolver for the `publishDate` field and resolve it with resolver in `Resolvers` class using the method `GetFormattedDate`
            2. The code ties the resolver to an actual field in an actual object type
                1. `publishDate` field in `Book` object type

    3. Inform GraphQL server that it needs to use `BookType` definition
        1. Program.cs

```
builder.Services....
    .AddType<BookType>();
```

    4. Debug using breakpoing
        1. Resolvers.cs
            1. Put a breakpoint
        2. Restart app
            1. Run
                1. F5 to resume the code

Developing GraphQL Server in NodeJS

1. GraphQL Server in Node.js
    1. Popular GraphQL implementation server is Apollo
2. GraphQLCourse
    1. New folder: MyGraphQLNodeJS
        1. Open cmd
            1. New nodejs app: `npm init -yes`
                1. `package.json` file was created
            2. Install packages required for implementing GraphQL using Apollo: `npm install @apollo/server graphql`
            3. `code .`
                1. package.json

```
"type": "module",
"scripts": {
    "start": "node index.js"
}
```

                    1. `module` - sets the way JS uses external packages
            4. Download GraphQLNodeFiles.zip
                1. Copy and paste the following files in the folder
                    1. index.js
                    2. books.json
3. index.js
    1. Schema
    2. Parsing json

```
var books = JSON.parse(fs.readFileSync("books.json"));
console.log(`Books: ${books}`);
```

    3. Resolvers - defines what resolvers we have in the app

```
const resolvers = {
    Query: {
        books: (parent, args, contextValue, info) => {
            return books;
        }
    },
};
```

        1. `books` - resolver
        2. It defines what happens if we retrieve `books` field in the `Query` object

    4. Initialization of Apollo server and starting it

```
const server = new ApolloServer({
    typeDefs,
    resolvers,
});

const { url } = await startStandaloneServer(server, { listen: { port: 4000 } });

console.log(`Server listening at: ${url}`);
```

5. Run: `npm start`
    1. Copy the address: http://localhost:4000
        1. Playground is a little different
    2. Query:

```
{
    books {
        bookId
        name
    }
}
```

        1. `books` resolver is used to retrieve the data
            1. Arguments:
                1. Uncomment: `console.log(info)`
        2. Restart the app
        3. Run the query again
            1. VS Code
                1. info in terminal
                    1. Good for analysis

Defining Field Resolvers in NodeJS

1. Adding a new resolver

```
Query: {
    ...
    Book: {
        name: (parent, args, contextValue, info) => {
            var name = parent.name; // the book object type
            return `[${name}]`;
        }
    }
}
```

    1. Terminal: Restart the app
    2. Playground:
        1. Run the query
            1. It displays with square brackets

Section 11: Developing GraphQL Client

Introduction

1. How to develop a graphql client and how to use it
    1. It is extremely simple
        1. We can use plain old JavaScript
            1. No special libraries are required
                1. There are some clients that can be used for advanced capabilities but in most cases, there's no need

Developing GraphQL Client

1. For .NET server (which is same for Node.JS server)
2. VS Code
    1. New folder: wwwroot
    2. Program.cs

```
app.UseStaticFiles(); // by default static files are served from `wwwroot` folder
```

3. Download: client.html
    1. Copy it to `wwwroot` folder
        1. client.html
            1. `runQuery()`
                1. uses `fetch` api (standard JS method)
                    1. `JSON.stringify` is used to send the body
4. Hit F5
5. Run: http://localhost:7293/client.html
    1. Hit the button
6. Modify the file:

```
function runQuery() {
    var name = document.getElementById("filter").value;
    var query = `query GetBooks($name: String) {
        books (nameContains: $name) {
            name
            pages
        }
    }`;
}
```

```
<body>
    <input id="filter" placeholder="Filter..."/>
```

7. Adding name variable to GraphQL (GraphQL server can use it)

```
fetch(...) {
    ...
    body: JSON.stringify({
        query,
        variables: {name}
    })
```

8. Open URL:
    1. Filter: H
    2. Run

Section 12: Advanced Topics

Introduction

1. So far, we have covered the most commonly-used features of GraphQL
    1. Might be enough to design and build a full-blown GraphQL-based API
        1. There are still some advanced topics that will help us make GraphQL even better
2. Topics:
    1. Error Handling
    2. Authorization

Error Handling

1. Newsflash:
    1. There will be errors in code and app
2. Errors in GraphQL are returned as JSON
    1. Code should plan for that and catch the errors
        1. Errors can be:
            1. Thrown automatically by the server when it cannot parse or validate the message
            2. Thrown by our own code
3. Either case - we should plan for it and act accordingly
    1. Usually - log the error and show generic message to the user
        1. How do the errors look like?
            1. VS Code
                1. Change `name` to `nameh` (it does not exist in the schema)
        2. Refresh browser tab
            1. Filter: H
            2. Run
                1. Error: JSON
        3. We change the code to handle the error
            1. client.html

```
fetch...
    .then(data => {
        if (date.errors) {
            console.error(data);
            document.getElementById("results").value = "An error has occured, and please try again later.";
        } else {
            document.getElementById("results").value = JSON.stringify(data, undefined, 4);
        }
    });
```

                1. `errors` collection is returned if there are errors

            2. Refresh the page
                1. Run
                    1. Open Console
                        1. Error is logged

Own Errors

1. client.html
    1. Fix the query
2. Query.cs

```
public class Query {
    public List<Book> Books<string nameContains = "") {
        ...
        throw new Exception("Error retrieving data");
        return ...
    }
    ...
}
```

    1. Server is going to throw exception when we try to read the books
        1. Restart server
    2. Refresh browser tab
        1. Run
            1. Console:
                1. Detailed explanation

1. Stack trace is shown
    1. We don't want this - a potential security hole
        1. Solution: Many GraphQL servers have their own way to indicate not to return stack trace (no security risks)
            1. .NET - run app without debugging
                1. HotChocolate GraphQL server will remove stacktrace and other problematic information
3. VS Code
    1. Run without debug (F5 executes with Debug capabilities)
        1. Ctrl + F5 (on click menu item)
    2. Refresh client
        1. Run
            1. Console: Expand error
                1. No extension containing stacktrace

Authorization

1. When running an API, the server needs to know who the user is, and what he's allowed to do
    1. This is called:
        1. Authentication (Who is the user)
        2. Authorization (What he is allowed to do)
2. **GraphQL offers no special treatment to authentication and authorization**
    1. It can be configured to pass the user to the resolvers
        1. **A resolver will decide what to do with this information**
3. **GraphQL does not care how the authentication is actually executed**

How to get user details in the resolver

1. We need to configure out app to work with Authentication
    1. Program.cs

        ```
        builder...
        builder.Services.AddAuthentication(); // indicates to app that it is going to use authentication
        ...
        app.UseAuthentication(); // indicates to use the authentication
        ```

        1. We didn't indicate which authentication to use
            1. AD
            2. Okta
            3. ...
2. We need to add a package to support passing user details to the resolver:
    1. Terminal
        1. `dotnet add package HotChocolate.AspNetCore.Authorization`
            1. Contains modules for passing user data to resolvers
3. Query.cs

    ```
    using System.Security.Claims;
    ...

    public List<Book> Books(ClaimsPrincipal claims, ...) {

    }
    ```

    1. `claims` - collection of attributes containing the user details
        1. When a user is authentication we can see in claims
            1. name
            2. email
            3. What he is allowed to do
            4. ...
        2. When a user is not authenticed, the claims collection will indicate that as well
    2. Place a breakpoint after method name
    3. Restart the app
        1. F5
            1. Playground:
                1. Run
                    1. Breakpoint will be hit
                        1. claims
                            1. Identities
                                1. IsAuthenticated: false
                                    1. Solution: We can add any authentication mechanism such as AD, Okta, own user management etc.
                                        1. Claims will return the user identity

Section 13: Conclusion

Conclusion

1. What we learned?
    1. GraphQL is an extremely flexible API
        1. It allows specifying the exact data we want to retrieve
        2. It is multi-platform
            1. .NET
            2. Node.JS
        3. It works with JSON
        4. It has sophisticated error handling information
2. We began from the very start
    1. Basics of API

    2. Web APIs
    3. GraphQL
3. We went all the way to the most advanced topics of GraphQL
4. We had a lot of hands-on
5. We had a lot of fun

Agenda

1. Welcome
2. API Basics
3. Web APIs
    1. GraphQL is part of
4. Introduction to GraphQL
    1. History
    2. Motivation
5. Preparing the Environment
6. GraphQL Schema
    1. Role
    2. Syntax
7. Queries
    1. Most common usage
8. Mutations
9. Subscriptions
10. Developing GraphQL Server
    1. .NET
    2. Node.JS
11. Developing GraphQL Client
    1. Using plaing JavaScript in browser
12. Advanced Topics
13. Conclusion

Bonus: Next Steps

1. Software Architects Discussions
2. Software Architecture Newsletter
3. Discounted Courses:
    1. Microsoft Azure: Cloud Architecture Case Studies
    2. Azure Security Best Practices
    3. Azure ChatGPT and OpenAI Service
    4. Azure IoT - The Complete Guide
    5. Microservices Security - The Complete Guide
    6. Building WebAPIs with gRPC - The Complete Guide
    7. Software Architecture Interview Questions
4. https://memilavi.com/